Computability - Recitation 1

March 11, 2026

1 Introduction

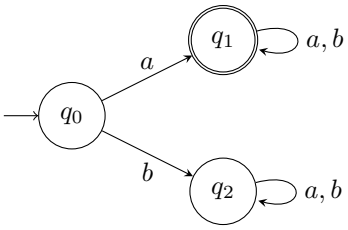
Who thinks that:

- You can write a python program that verifies that another python program always sorts an input array correctly?
- If you get a map and should decide whether you can color each country in one of three colors such that there is no border between two country with the same color, can you do it efficiently?

After learning this course you will know the answer to the first question. The second is an open question (100 in the course for whoever solves it).

2 Examples of deterministic finite automata

Consider the following DFA, which runs on input words made of a's and b's.

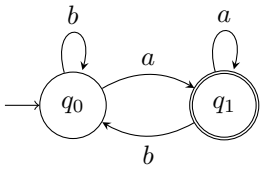


Let's run it on some words:

- On the word abb: we start at q_0. The first letter is a so we move to q_1, we see b and stay at q_1, see another b and finish the run at q_1. The word is accepted.
- On the word b: we start at q_0. We see b so we move to q_2. The word is rejected.
- The empty word: we start at q_0 and immediately stop. The word is rejected.

The words accepted by this automaton are exactly those that start with a.

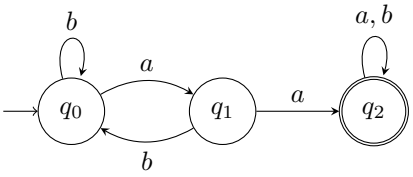
Another example:



- The word ab: we start at q_0. We see a so we move to q_1, then we see b so we move to q_0. The word is rejected.
- The word aba: start the same as before, but we see another a so we move to q_1 at the end. The word is accepted.
- The empty word: we start at q_0 and stop. The word is rejected.

The words accepted by this automaton are exactly those that end with a.

Another example: suppose we want an automaton that accepts exactly the words that contain aa (two consecutive a's). Intuitively, we want to remember two things: whether aa has ever appeared (in which case we should accept, no matter what came after it); and if not, whether the previous letter is a (so that if we see another a, we know to accept).



In order to write a formal proof that the automaton accepts exactly the words we want:

1. Identify which words end in which states. In the above example, the states can be characterized as follows:
   - q_0: words that don't contain aa, and don't end with a.
   - q_1: words that don't contain aa, and end with a.
   - q_2: words that contain aa.
2. Prove by induction on the word length. In the induction step, show that every transition preserves the states' characterizations.

3 Formal Definitions

3.1 Languages

Let Σ be a finite (non-empty) set, which we will call *alphabet*. The elements of Σ are called *letters*.

Example: $\Sigma = \{a, b\}$

We now consider sequences of letters. For every $n = 1, 2, \ldots$ define:

$$\Sigma^n = \{(\sigma_1, \ldots, \sigma_n) : \sigma_1, \ldots, \sigma_n \in \Sigma\}$$

For $n = 0$, we define $\Sigma^0 = \{\epsilon\}$, where ϵ is an "empty sequence".

Example: $\Sigma^3 = \{(a, a, a), (a, a, b), (a, b, a), \ldots, (b, b, b)\}$.

Define:

$$\Sigma^* = \bigcup_{n=0}^{\infty} \Sigma^n$$

The elements of Σ^* are sequences of any finite length of letters from Σ, and we refer to those as *words*. ϵ is called the *empty word*. For convenience, we don't write words as (a, b, b, b, a), but rather as $abbba$.

A *formal language* over the alphabet Σ is a set $L \subseteq \Sigma^*$. That is, a set of words.

Example:
- $L_1 = \{\epsilon, a, aa, b\}$. A finite language.
- $L_2 = \{w : w \text{ starts with an } a\}$. An infinite language.
- $L_3 = \{\epsilon\}$.
- $L_4 = \emptyset$. Is this the same as L_3?
- $L_5 = \{w : |w| < 24\}$. $|w|$ is the number of letters in w. Finite language.

3.2 DFA

We have an intuitive understanding of what a DFA is and how it operates: start at q_0, change states according to each input letter, and check whether the last state is accepting. This was sufficient to work through the above examples. However, there are some properties of DFAs that require more formalism. For example, if

$L_1, L_2 \subseteq \Sigma^*$ have DFAs $\mathcal{A}_1, \mathcal{A}_2$, can we construct a DFA for $L_1 \cap L_2$? We will see later that the answer is yes. The proof will require us to work with a precise definition.

To define a DFA formally we need:

1. A finite set of states, Q.
2. A finite alphabet, Σ.
3. A transition function, $\delta : Q \times \Sigma \to Q$, which tells us what state to go to based on the current state and input letter.
4. An initial state, $q_0 \in Q$.
5. A set of accepting states, $F \subseteq Q$.

A DFA is defined as a 5-tuple: $\mathcal{A} = \langle Q, \Sigma, \delta, q_0, F \rangle$. We now define formally its operation on a given input. Let $w = w_1 w_2 \dots w_n \in \Sigma^*$ be a word with n letters. $\mathcal{A}$'s run on w is a sequence of states, $r_0, r_1, \dots, r_n \in Q$ such that:

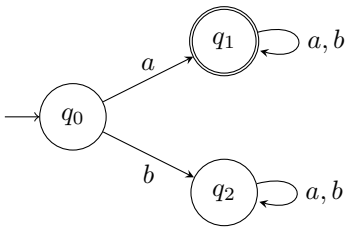
- Initial state: $r_0 = q_0$.
- Transitions: for every $0 \leq i < n$, we have $r_{i+1} = \delta(r_i, w_{i+1})$.

We say that $\mathcal{A}$ *accepts* w iff the last state r_n is in F. The language of the automaton is the set of accepted words:

$$L(\mathcal{A}) = \{w \in \Sigma^* : \mathcal{A} \text{ accepts } w\}$$

We also say that $\mathcal{A}$ *decides* $L(\mathcal{A})$.

Example: Consider the DFA from before:



Formally, $\mathcal{A} = \langle Q, \Sigma, \delta, q_0, F \rangle$ where:

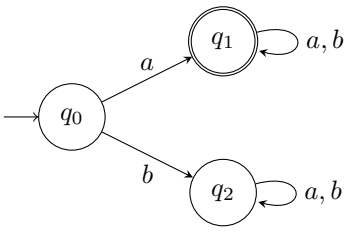
- The set of states is $Q = \{q_0, q_1, q_2\}$.
- The alphabet is $\Sigma = \{a, b\}$.
- The transition function δ is defined as:

	a	b
q_0	q_1	q_2
q_1	q_1	q_1
q_2	q_2	q_2

- The initial state is q_0.
- The set of accepting states is $F = \{q_1\}$.

4 Formally proving what the language of an automaton is

In class, you have seen several DFAs, and mentioned what their language is. How can we be sure, however, that the language we think they decide is actually the language they decide? "It's obvious", as you know, is a dangerous expression in mathematics. When we say that something is obvious, we mean that if we are asked to, we can very easily provide the proof. In this section, we will see how to prove the correctness of a language conjecture. This is a rather painful procedure, and we will do it once here, and once in the exercise. After that, we will allow you to explain less formally why an automaton decides a language. However, whenever you claim that a certain language is decided by an automaton, always make sure you could prove it formally at gun point. Consider the automaton from the example we saw.



We argue that

$$L(\mathcal{A}) = L$$

where

$$L := \{w \in \Sigma^* \mid w \text{ begins with } a\}.$$

We claim that the words that end in each state are as follows:

- q_0: the empty word.
- q_1: words that start with a.
- q_2: words that don't start with a and are not the empty word.

The claim is sufficient: $w \in L \Leftrightarrow w$ starts with $a \Leftrightarrow \mathcal{A}$ ends in its run on w in $q_1 \Leftrightarrow \mathcal{A}$ accepts w (since q_1 is the only accepting state) $\Leftrightarrow w \in L(\mathcal{A})$.

We prove the claim by induction on $|w|$.

Base case: $|w| = 0$. Then w is the empty word ϵ. The run of $\mathcal{A}$ on ϵ ends in q_0, and indeed ϵ does not start with a.

Step: Assume that the claim is true for words of length n for some $n \geq 0$. Consider a word $w = u\sigma$, where $|w| = n + 1$ and $|u| = n$. Split to cases according to σ and the state $\mathcal{A}$ reaches on u:

- q_0: By the induction hypothesis, u is the empty word. If $\sigma = a$, then $w = u\sigma = \epsilon a = a$ starts with a. So the run of $\mathcal{A}$ on w should end in q_1, and indeed, $\delta(q_0, a) = q_1$. If $\sigma = b$, then $w = u\sigma = \epsilon b = b$ does not start with a and is not the empty word. So the run of $\mathcal{A}$ on w should end in q_2, and indeed, $\delta(q_0, b) = q_2$.
- q_1: By the induction hypothesis, u starts with a. If $\sigma = a$, then $w = u\sigma$ which starts with a. So the run of $\mathcal{A}$ on w should end in q_1, and indeed, $\delta(q_1, a) = q_1$. If $\sigma = b$, then $w = u\sigma$ which starts with a. So the run of $\mathcal{A}$ on w should end in q_1, and indeed, $\delta(q_1, b) = q_1$. (Notice we could prove without splitting to cases)
- q_2: By the induction hypothesis, u does not start with a and is not the empty word. So the run of $\mathcal{A}$ on w should end in q_2, and indeed, $\delta(q_2, \sigma) = q_2$ for every $\sigma \in \Sigma$.

5 The extended transition function δ^*

Let $\mathcal{A} = \langle Q, \Sigma, \delta, q_0, F\rangle$. The transition function $\delta : Q \times \Sigma \to Q$ captures our intuition of reading a letter and moving to the next state in $\mathcal{A}$. We also have an intuitive notion of feeding an entire word as input, and moving to the appropriate state after applying a transition for each letter. We define a function δ^*, whose input is a state and a word, and its output is the resulting state. Formally, $\delta^* : Q \times \Sigma^* \to Q$ is defined recursively:

$$\delta^*(q, w) \stackrel{\text{def}}{=} \begin{cases} q & w = \epsilon \\ \delta(\delta^*(q, w'), \sigma) & w = w'\sigma \end{cases}$$

That is, if the word w is empty, we stay in q. Otherwise, w can be written as $w = w'\sigma$ for some letter $\sigma \in \Sigma$ and a shorter word $w' \in \Sigma^*$. We apply δ^* on the shorter word, and then δ for the last transition.

Let's prove a claim about δ^*.

Proposition 1. *Let $\mathcal{A} = \langle Q, \Sigma, \delta, q_0, F\rangle$. For every $q \in Q$ and $w, w' \in \Sigma^*$, we have:*

$$\delta^*(q, w \cdot w') = \delta^*(\delta^*(q, w), w')$$

Intuition: if we start at some state q, and move along the transitions as we read the input $w \cdot w'$, this is the same as first reading w and then, from the resulting state, reading w'. This may sound obvious, but proving it is a good exercise for working formally with automata.

Proof. The proof is by induction on $|w'|$, the length of the word w'.

Base case: Let $w' = \epsilon$. Then:

$$\delta^*(\delta^*(q, w), w') \overset{(1)}{=} \delta^*(\delta^*(q, w), \epsilon) \overset{(2)}{=} \delta^*(q, w) \overset{(3)}{=} \delta^*(q, w \cdot w')$$

In (1) we substitute $w' = \epsilon$. (2) is by the definition of δ^* for the word ϵ. (3) holds because $w = w\epsilon = w \cdot w'$.

Induction step: Assume that the claim holds for words of length n, for some $n \geq 0$, and consider a word $w' = w''\sigma$ of length $n + 1$, where $\sigma \in \Sigma$ is a letter. We have:

$$\delta^*(q, w{\cdot}w') \overset{(1)}{=} \delta^*(q, w{\cdot}w''{\cdot}\sigma) \overset{(2)}{=} \delta(\delta^*(q, w{\cdot}w''), \sigma) \overset{(3)}{=} \delta(\delta^*(\delta^*(q, w), w''), \sigma) \overset{(2)}{=} \delta^*(\delta^*(q, w), w''\sigma) \overset{(1)}{=} \delta^*(\delta^*(q, w), w')$$

In (1) we use the fact $w' = w''\sigma$. (2) is by the definition of δ^* for a non-empty word. (3) follows from the induction hypothesis for the word w''. □

Definition 2 (characterization of DFA via δ^*)**.** *The language of a DFA $\mathcal{A} = \langle Q, \Sigma, \delta, q_0, F\rangle$ is*

$$L(\mathcal{A}) = \{w \in \Sigma^* : \delta^*(q_0, w) \in F\},$$

that is, a word $w \in \Sigma^*$ is accepted by $\mathcal{A}$ if and only if $\delta^*(q_0, w) \in F$.

6 Regularity of $L_{\mathbf{EVEN}}$

6.1 Regular languages

It is often useful to have terminology for a collection of languages that have a useful property.

Definition 3 (regular languages and REG)**.** *A language $L \subseteq \Sigma^*$ is called* regular *if there is a* DFA *$\mathcal{A}$ that decides it. The collection of regular languages is called* REG*:*

$$\text{REG} \overset{\text{def}}{=} \{L \subseteq \Sigma^* : \textit{there is a DFA } \mathcal{A} \textit{ that decides } L\}$$

Question: Let $L \in$ REG and let $\mathcal{A} = \langle Q, \Sigma, \delta, q_0, F\rangle$ be the DFA that decides L.

Consider the language:

$$L_{\text{EVEN}} = \{w \in L \mid |w| \text{ is even}\}.$$

Is L_{EVEN} also regular?

Answer: Yes!

6.2 Intuition

Since L is regular, it is recognized by some DFA. To decide L_{EVEN}, we need to ensure that a word belongs to L and has even length. This can be achieved by constructing a modified DFA that tracks the length's parity along with the original state transitions.

6.3 Proof Sketch

Let $\mathcal{A} = \langle Q, \Sigma, \delta, q_0, F\rangle$ be a DFA that decides L. To construct a DFA for L_{EVEN}, we define a new automaton $\mathcal{A}'$ that tracks both the state of $\mathcal{A}$ and the parity of the length of the input string.

Define $\mathcal{A}' = \langle Q', \Sigma, \delta', (q_0, 0), F'\rangle$, where:

- $Q' = Q \times \{0, 1\}$, where the second component tracks the parity (0 for even, 1 for odd).
- The transition function δ' is defined as:

$$\delta'((q, p), a) = (\delta(q, a), 1 - p), \quad \forall q \in Q, p \in \{0, 1\}, a \in \Sigma.$$

- The initial state is $(q_0, 0)$.
- The set of accepting states is $F' = \{(q, 0) \mid q \in F\}$, ensuring acceptance only when the word length is even and the state corresponds to an accepting state in $\mathcal{A}$.

Since $\mathcal{A}'$ is a DFA, the language it accepts, L_{EVEN}, is regular, proving the claim.